





INTERPRÉTEUR DE COMMANDES « SHELL »

Mr Ahmed KHARRAT
Système d'exploitation
Filière: Génie Informatique / BI
2025-2026

INTRODUCTION

2

- Le **shell** (coquille qui entoure le "noyau" du système) est **un interpréteur de commandes**, c'est-à-dire un programme capable de comprendre et d'exécuter un ensemble de commandes paramétrables, **qui sert d'intermédiaire entre l'utilisateur et le système d'exploitation**.
- Il existe plusieurs shells pour LINUX, essentiellement Bourne Again Shell ou BASH et TCSH. Nous étudierons le **BASH**.

Ligne de commande

« command line »

3

- La ligne de commande est ce qui est envoyé à l'interpréteur de commandes en réponse à la touche ENTER.
- Cette ligne est composée du **nom de la commande** qui peut être le nom d'un fichier exécutable ou une commande du « shell », et **d'arguments**, s'il y a lieu.
- La syntaxe est la suivante : ***command [argument1] [argument2] [argument3]...[argumentN] ENTER***
- Les crochets indiquent que les arguments sont optionnels. La commande et ses arguments sont séparés entre eux par une ou plusieurs espaces ou tabulations.

© Ahmed KHARRAT

Les groupes de commandes

4

Il existe plusieurs méthodes pour enchaîner des commandes sur une même ligne :

- Exécution séquentielle :

cmd1; cmd2;...; cmdN

- Exécution sous condition d'erreur :

cmd1 || cmd2 || ... || cmdN

si cmd1 ne se termine pas correctement, alors cmd2 est exécuté, et ainsi de suite.

- Exécution sous conditions de réussite :

cmd1 && cmd2 && ... && cmdN

si cmd1 s'est bien déroulée, alors cmd2 sera exécutée; et ainsi de suite.

© Ahmed KHARRAT

Caractères spéciaux (1)

5

Les caractères suivants ont une signification particulière pour l'interpréteur de commandes.

- **Le caractère «?»** : L'interpréteur de commande remplace ce symbole par **exactement un caractère**.
- **Le caractère «*»** : L'interpréteur de commande remplace ce symbole par **aucun, un ou plusieurs caractères**.
- **Les caractères «[]»** : Ce symbole est plus restrictif que « ? » car il demande à l'interpréteur de le remplacer par **exactement un seul des caractères spécifiés entre les crochets**.
- **Le caractère «\»** : Ce caractère indique à l'interpréteur de commande **ne pas interpréter le prochain caractère comme un caractère spécial**.

© Ahmed KHARRAT

Caractères spéciaux (2)

6

Par exemple,

La chaîne	sera interprétée comme
*	*
\ ?memo*	?memo*

- **Le caractère «"»** : Ce caractère a la même fonction que le caractère \. La seule différence est que cette double apostrophe doit être utilisée par paire.
- **Le caractère «>»** : Redirige la sortie standard d'un programme **dans un fichier au lieu de l'écran**.
- **Le caractère «<»** : Redirige l'entrée standard d'un programme **à partir d'un fichier au lieu du clavier**.
- **Le caractère « | »** : C'est le symbole du tube (**« pipe »**) qui sera vu dans la section suivante:

© Ahmed KHARRAT

Caractères spéciaux (3)

7

- Pour rediriger le résultat d'une commande dans le flux d'entrée d'une autre commande, il faut utiliser l'opérateur '|'. **Cet opérateur représente un tuyau canalisant les données issues d'une commande vers le flux d'entrée standard de la commande suivante**, d'où le nom de « pipe » (« tuyau » ou « tube » en anglais). L'opérateur pipe s'utilise de la manière suivante :
 - on écrit la première commande, qui doit fournir les données à la deuxième commande ;
 - on écrit l'opérateur pipe ;
 - on écrit la deuxième commande, qui doit lire les données provenant de la première

© Ahmed KHARRAT

Caractères spéciaux (4)

8

- **Le caractère «&»** : Ce caractère indique à l'interpréteur de faire **l'exécution de la commande qui précède en arrière-plan (« background »)**.
- **Le caractère «\$»** : Ce caractère indique à l'interpréteur que le mot qui suit **est le nom d'une variable d'environnement et qu'il faut utiliser son contenu**.
- **Le caractère «~»** : Ce caractère référence **un répertoire privé quelconque**. Par exemple, «~heidi» signifie le répertoire privé de l'utilisateur heidi.
- **Les caractères «>>»** : Redirigent la sortie standard d'un programme dans un fichier au lieu de l'écran. **Cependant, cette fois-ci le fichier sera augmenté des données (concaténation) sortant du programme au lieu d'être écrasé (s'il existe)**.

© Ahmed KHARRAT

Gestion des utilisateurs et des groupes (1)

9

Commande	sera interprétée comme
useradd, usermod, userdel useradd -m nom_utilis	gestion des comptes utilisateur Le paramètre -m indique à la commande useradd de créer le répertoire personnel de l'utilisateur dans le répertoire /home.
groupadd, groupmod, groupdel	gestion des groupes
Passwd	changer le mot de passe

Exemple :

- Créer un nouveau groupe nommé stagiaire.

groupadd stagiaire

- Supprimer le groupe totox

groupdel totox

© Ahmed KHARRAT

Gestion des utilisateurs et des groupes (2)

10

- Pour créer l'utilisateur **stagex**, root passe la commande :

useradd stagex

- Pour lui attribuer le mot de passe :

passwd stagex

La commande **passwd** est chargée aussi du cryptage du mot de passe dans /etc/shadow

Syntaxe : **passwd [option] nom-login**

- ▣ **-d**, pour supprimer le mot de passe, l'utilisateur pourra se connecter sans !
- ▣ **-l**, pour verrouiller le compte et empêcher sa connexion.
- ▣ **-u**, pour déverrouiller.

© Ahmed KHARRAT

Gestion des utilisateurs et des groupes (3)

11

- Supprimer le compte d'un utilisateur (non connecté), au hasard ..
totox.

userdel -r totox

L'option -r supprime toute trace de l'utilisateur dans le fichier de configuration : /etc/passwd y compris dans les groupes d'utilisateurs.

- Modifier le compte de l'utilisateur stagex

Syntaxe : **usermod [options] totox**

Les options sont les mêmes que **useradd**

usermod -G stagiaire,prof stagex

stagex devient membre des 2 groupes *stagiaire* et *profs* (à l'exclusion de tout autre groupe secondaire).

Commandes de base (internes) man

12

A. la commande **man** : L'aide en ligne

- **Syntaxe:** **man** [section] commande
- **Description:** Elle permet de consulter le manuel du Shell à propos d'une commande.

Les pages **man** sont organisées en 8 sections standard:

1 = Commandes utilisateur

.....

4 = Format des fichiers système.

8 = Procédures de maintenance système.

Commandes de base (internes)

man

13

□ Options courantes:

Section: Numéro de la section qui contient la page.

□ Exemple :

man passwd

man 4 passwd

© Ahmed KHARRAT

Commandes de base (internes)

ls

14

B. la commande ls:

- **Syntaxe:** **ls** [options] [noms]
- **Description:** **ls** liste les répertoires et les fichiers précisés dans noms. Sans options, **ls** n'envoie que le nom des fichiers. Si noms n'est pas précisé, c'est le répertoire courant qui est listé.
- **Options courantes:**
 - ▣ **-a**, Tous les fichiers (y compris les fichiers cachés ceux qui commencent par un point)
 - ▣ **-d**, Affiche le nom des répertoires sans leur contenu.
 - ▣ **-l**, Format long (avec beaucoup de détails).
 - ▣ **-r**, Affiche tous les fichiers triée en ordre décroissant
 - ▣ **-R**, Les répertoires de sous-répertoires dans l'ordre inverse.

© Ahmed KHARRAT

Commandes de base (internes)

ls

15

□ Exemple :

ls -l programmes

- donne le contenu du répertoire «programmes » en donnant des informations sur les fichiers (taille, dates, ...)

© Ahmed KHARRAT

Commandes de base (internes)

cd

16

C. la commande **cd**:

- **Syntaxe:** **cd** [répertoire]
- **Description:** La commande **cd** permet de changer le répertoire de travail. Si répertoire n'est pas précisé, alors le nouveau répertoire de travail sera le répertoire de connexion (\$HOME).
- **Options courantes:**
 - ▣ **répertoire:** représente le futur répertoire de travail.
- **Exemple :**

```
cd /usr/lib/news/bin
cd $HOME/rep1
```

© Ahmed KHARRAT

Commandes de base (internes)

cd

17

❑ Remarque :

- ❑ la commande **cd**, comme toutes les commandes utilisant des répertoires, permet de spécifier deux types de chemins :
 - **les chemins relatifs** : ils sont relatifs au répertoire de travail, et utilisent notamment le répertoire **'..'** (répertoire père).

❑ Exemple 1 : **cd ../rep1**

- **les chemins absolus** : ils faut spécifier toute l'arborescence depuis la racine.

❑ Exemple 2 : **cd /home/rep1**

© Ahmed KHARRAT

Commandes de base (internes)

cd

18

Exemple 3:

- ❑ Le chemin d'accès absolu à mon répertoire personnel est **/home/dupont** ; j'ai créé plusieurs sous-répertoires de telle sorte que les chemins **/home/dupont/doc/cours/info** et **/home/dupont/pgs** sont valides. Comment aller dans le second répertoire à partir du premier ? Il y a plusieurs solutions :

- ❑ **cd .. , cd .. , cd .., cd pgs** (on remonte de 3 niveaux, on est dans **~ username**, on va dans **pgs**) ;
- ❑ **cd ../../../pgs** (la même solution condensée en une seule commande) ;
- ❑ **cd ~/pgs** ;
- ❑ **cd /home/dupont/pgs.**

© Ahmed KHARRAT

Commandes de base (internes)

pwd

19

D. la commande **pwd**: Savoir où on est

- **Syntaxe:** **pwd**
- **Description:** **pwd** (*print working directory*) retourne le chemin absolu du répertoire courant.

□ **Options courantes:**

La commande **pwd** n'accepte pas d'option.

□ **Exemple**

pwd

/usr/lib/news/bin

© Ahmed KHARRAT

Commandes de base (internes)

mkdir

20

E. la commande **mkdir** :

- **Syntaxe:** **mkdir** [-p] nouveau_répertoire
- **Description:** **mkdir** (*make directory*) permet de créer un répertoire.
- **Options courantes:**
 - ▣ **-p** permet de créer tous les répertoires intermédiaires qui n'existeraient pas,
 - ▣ **nouveau_répertoire** le nom du répertoire à créer. C'est un argument obligatoire.

© Ahmed KHARRAT

Commandes de base (internes)

mkdir

21

Exemple :

```
mkdir rep1
```

```
cd rep1
```

```
mkdir rep11/rep111
```

```
mkdir: Cannot create directory " rep11/rep111": No such file or directory
```

solution : **mkdir -p rep11/rep111**

Remarque :

Pour pouvoir créer un répertoire, le répertoire d'origine doit avoir les droits en écriture positionnés.

© Ahmed KHARRAT

Commandes de base (internes)

rmdir

22

F. la commande **rmdir**: Détruire un répertoire vide

- **Syntaxe:** **rmdir** [-p] [-s] répertoire
- **Description:** **rmdir** (**remove directory**) ne permet de détruire un répertoire que s'il est vide. Pour détruire un répertoire non vide, utiliser **rm -R**.
- **Options courantes:**
 - ▣ **-p** permet de détruire tous les sous-répertoires vides.
 - ▣ **-s** mode silencieux (aucun affichage).
 - ▣ **répertoire** représente le nom du répertoire à détruire. C'est un argument obligatoire.

© Ahmed KHARRAT

Commandes de base (internes)

rmdir

23

Exemple :

```
rmdir /tmp/rep1/rep11/rep111
cd /tmp
rmdir -p rep1/rep11
rmdir: rep1/rep11: Whole path removed.
cd rep1
rep1: does not exist
```

Remarque :

Pour pouvoir supprimer un répertoire, le répertoire père doit avoir les droits en écriture positionnés. Voir aussi la commande **rm -r** pour supprimer des répertoires contenant des fichiers.

© Ahmed KHARRAT

Commandes de base (internes)

rm

24

G. la commande **rm**:

- **Syntaxe:** **rm** [-f] [-i] [-r] fichier1/répertoire1 [fichier2/répertoire2...]
- **Description:** **rm (remove)** permet de supprimer des fichiers ou des répertoires.
- **Options courantes:**
 - ▣ **-i** mode interactif, demande la confirmation avant écrasement.
 - ▣ **-f** force la commande.
 - ▣ **-r** récursif (détruit tous les sous répertoires. ATTENTION)

© Ahmed KHARRAT

Commandes de base (internes)

rm

25

Exemple :

```
rm -i fic*
```

```
rm: File fic1. Remove ( yes/no)? y
```

```
rm: File fic2. Remove ( yes/no)? N
```

Remarque :

Pour pouvoir supprimer un fichier, le répertoire où se trouve le fichier doit avoir ses droits en écriture positionnés ; sauf dans le cas où le répertoire aurait les droits suivants :

```
drwxrwxrwt 1 user user 6 Nov 21 2015 rep1
```

© Ahmed KHARRAT

Commandes de base (internes)

cp

26

H. la commande **cp**:

- **Syntaxe:** **cp** [-i] [-p] fichier1 fichier2
cp [-i] [-p] [-r] source1 [source2...] répertoire
- **Description:** cp (**copy**) copie le contenu de fichier1 dans fichier2 ; ou bien elle copie source1 et source2 (etc...) dans répertoire.
- **Options courantes:**
 - ▣ **-i** mode interactif, demande la confirmation avant écrasement.
 - ▣ **-p** conserve les dates du fichier source.
 - ▣ **-r** copie récursive de répertoires.
 - ▣ **source X** représente le nom des fichiers ou répertoires à copier.

© Ahmed KHARRAT

Commandes de base (internes)

cp

27

Exemple :

cp toto.c exemple.c

créer le fichier « toto.c » identique au fichier « exemple.c »

Remarque :

Pour pouvoir copier un fichier/répertoire, vous devez avoir les droits suivants :

- droits de lecture du fichier à copier ;
- droits d'exécution sur le répertoire contenant le fichier à copier ;
- droits d'écriture sur le répertoire de destination.

© Ahmed KHARRAT

Commandes de base (internes)

mv

28

I. la commande **mv**:

- **Syntaxe:** **mv** [-f] [-i] source1 [source2...] destination
- **Description:** **mv** (**move**) déplace les fichiers fichier1, fichier2 etc...dans destination.
- **Options courantes:**
 - ▣ **-i** mode interactif, demande la confirmation avant écrasement.
 - ▣ **-f** force la commande.
 - ▣ **Source X** représente le nom des fichiers ou répertoires à déplacer.
 - ▣ **destination** représente le nom des fichiers ou répertoires de destination.

© Ahmed KHARRAT

Commandes de base (internes)

mv

29

Exemple :

1. **mv** toto.txt toto2.txt

la commande **mv toto.txt toto2.txt** a pour effet de renommer le fichier toto.txt en toto2.txt.

2. **mv** toto.txt toto2.txt rep1

la commande **mv toto.txt toto2.txt rep1** a pour effet de déplacer les fichier toto.txt et toto2.txt dans rep1.

Remarque :

Pour pouvoir copier un fichier/répertoire, le répertoire cible doit avoir les droits en écriture positionnés, les droits en lecture sur le fichier source, et les droits d'accès dans le répertoire source.

© Ahmed KHARRAT

Commandes de base (internes)

cat et tac

30

J. les commandes **cat** et **tac** :

- **Syntaxe:** cat [fichier...]
- tac [fichier...]
- **Description:** cat et tac permettent d'afficher sur la sortie standard l'entier contenu d'un fichier.
- **Exemples:**
 - ▣ Exemple 1 : **cat** essai.doc
 - ▣ Exemple 2 : **tac** essai.doc
- L'exemple 1 affiche à l'écran le contenu du fichier essai.doc,
- l'exemple 2 est homologue à cat mais affiche le contenu d'un fichier en partant de la dernière ligne vers la première.

© Ahmed KHARRAT

Commandes de base (internes)

cat et tac

31

- **Description:** La commande **cat** permet aussi de concaténer des fichiers et d'en renvoyer le résultat sur la sortie standard.
- **Syntaxe:** `cat fichier_1 fichier_2 fichier_3`
- **Exemple:**
 - ▣ `cat lettre.txt rapport.txt conclusion.txt`
- L'exemple précédent affiche à l'écran le contenu des fichiers *lettre.txt*, *rapport.txt* et *conclusion.txt*.

© Ahmed KHARRAT

Commandes de base (internes)

cat et tac

32

- **Description:** Au lieu d'afficher sur la sortie standard le résultat d'une concaténation, on peut la rediriger vers un fichier.
- **Syntaxe:** `cat fichier_1 fichier_2 fichier_3 > fichier_4`
- **Exemple:**
 - ▣ `cat lettre.txt rapport.txt conclusion.txt > publication.txt`
- Dans cet exemple le fichier *publication.txt* aura pour contenu la concaténation des fichiers *lettre.txt*, *rapport.txt* et *conclusion.txt*.

© Ahmed KHARRAT

Commandes de base (internes) head et tail

33

K. les commandes **head** et **tail** :

- **Syntaxe:** **head** [-n] [fichier...]
- **tail** [-n | +n] [-f] [fichier]
- **Description:** La commande **head** affiche les n premières lignes d'un fichier, alors que **tail** affiche les dernières lignes d'un fichier. Si n n'est pas précisé, il prend la valeur 10.
- **Options courantes:**
 - ▣ **-n** nombre de lignes à afficher depuis le début/la fin de fichier.
 - ▣ **+n** affichage à partir de la ligne numéro n.
 - ▣ **-f** attente de nouvelles lignes (sortie par Ctrl-c).
 - ▣ **fichier** nom des fichiers à traiter.

© Ahmed KHARRAT

Commandes de base (internes) head et tail

34

Exemple :

1. **head** exemple.c
affiche les 10 premières lignes du fichier exemple.c
2. **tail** -5 exemple.c
affiche les 5 dernières lignes du fichier exemple.c
3. **head** -20 fichier
affiche les 20 premières lignes de fichier
4. **tail** - 20 fichier
affiche les 20 dernières lignes de fichier
5. **tail** + 8 fichier
affiche à partir du huitième ligne de fichier

© Ahmed KHARRAT

Commandes de base (internes)

touch

35

L. la commande **touch**:

- **Syntaxe:** **touch** [option] fichier
- **Description:** Modifie la caractéristique d'un fichier s'il existe ou créer un fichier vide s'il n'existe pas déjà.
- **Options courantes:**
 - **-a** : dernier accès seulement
 - **-m** : dernière modification seulement
- **Exemple:**
touch exemple.c
- Dans cet exemple **touch** modifie la date de dernier accès au fichier « exemple.c » à la date courante.

© Ahmed KHARRAT

Commandes de base (internes)

find

36

M. la commande **find**:

- **Syntaxe:** **find** répertoire option1 [option2...]
- **Description:** **find** permet de rechercher un fichier dans l'arborescence à partir du point spécifié.
- **Options courantes:**
 - **-name fich** recherche sur le nom fich
 - **size n** recherche sur la taille en blocs
 - **-ctime n** recherche sur la date de création
 - **-type d/f** : rechercher des répertoires (d) ou des fichiers (f).
 - **!** négation du critère de recherche

Exemple:

find HOME - name "vil*"

© Ahmed KHARRAT

Commandes de base (internes)

grep

37

M. la commande **grep**:

- **Syntaxe:** `grep [-ilsv] expression [fichier...]`
 - **Description:** **grep** permet de rechercher expression dans fichier. Elle affiche les noms de fichiers ainsi que les lignes contenant expression.
 - **Options courantes:**
 - ▣ **-i** ne tient pas compte des minuscules et des majuscules.
 - ▣ **-l** n'affiche que le nom des fichiers (pas les lignes).
 - ▣ **-n** affiche avec numéro de ligne.
 - ▣ **-f** fich spécifie un fichier contenant les expressions à rechercher.
 - ▣ **-v** affiche toutes les lignes, sauf celles qui contiennent l'expression
- expression chaîne de caractères (ou expression régulière non abordée dans ce cours) fichier nom des fichiers à traiter.

© Ahmed KHARRAT

Commandes de base (internes)

grep

38

Exemple :

grep -n -i « void » exemple.c

affiche, avec leurs numéros, les lignes du fichier « exemple.c »

qui contiennent la chaîne « void » en ne tenant pas compte des majuscules et des minuscules

© Ahmed KHARRAT

Commandes de base (internes)

ln

39

N. la commande ln:

- **Syntaxe:** `ln [-s] nom-du-fichier-réel nom-du-lien-symbolique`
- **Description:** `ln` permet la création d'une nouvelle référence pour un fichier préexistant. Les liens symboliques représentant des pointeurs virtuels (raccourcis) vers des fichiers réels. En cas de suppression du lien symbolique le fichier pointé n'est pas supprimé.
- **Options courantes:**
 - ▣ `-s` permet de faire un lien symbolique
- **Exemple :**
 - ▣ `ln -s exemple.c toto.c` crée un lien symbolique sur le fichier « `exemple.c` , de nom « `toto.c` »

© Ahmed KHARRAT

Commandes de base (internes)

nl

40

O. la commande nl:

- **Syntaxe:** `nl fichier`
- **Description:** `nl` permet l'affichage du contenu d'un fichier et en numérote les lignes.
- **Exemple :**
 - ▣ `nl lettre.doc`

Cette commande permet d'afficher le contenu du fichier « `lettre.doc` » en numérotant ces lignes.

© Ahmed KHARRAT

Commandes de base (internes)

sort

41

P. la commande **sort**:

- **Syntaxe:** **sort** [-bdfnr] *fichier*
- **Description:** **sort** permet de trier les lignes d'un fichier.
- **Options courantes:**
 - ▣ **-b** ignore les espaces en début de ligne
 - ▣ **-d** ordre alphabétique (A-Z, a-z, 0-9, espace) (par défaut)
 - ▣ **-f** ignore la casse
 - ▣ **-n** ordre numérique
 - ▣ **-r** inverse l'ordre de tri.

© Ahmed KHARRAT

Commandes de base (internes)

sort

42

Exemple :

sort -bdf *essai.txt*

Cet exemple permet de trier les lignes du fichier *essai.txt* dans l'ordre alphabétique (d) sans tenir compte des espaces de début de ligne (b) et sans différencier majuscules et minuscules (f).

© Ahmed KHARRAT

Commandes de base (internes)

wc

43

Q. la commande **wc**:

- **Syntaxe:** **wc** [-lwc] [fichier...]
- **Description:** **wc** compte le nombre de lignes, mots, ou caractères d'un fichier texte. Si aucun fichier n'est passé en paramètre, c'est l'entrée standard qui sera lue. Si aucune option (-lwc) n'est précisée, alors **wc** compte le nombre de lignes, mots, **et** caractères du fichier.
- **Options courantes:**
 - ▣ **-l** précise que c'est le nombre de lignes qui doit être compté.
 - ▣ **-w** précise que c'est le nombre de mots qui doit être compté.
 - ▣ **-c** précise que c'est le nombre de caractères qui doit être compté.

© Ahmed KHARRAT

Commandes de base (internes)

wc

44

Exemple :

1. **wc -l villes.doc**

Résult: 4 villes

Le fichier « villes.doc » contient 4 lignes.

2. **wc villes.doc**

Résult: 4 4 36 villes

Le fichier « villes.doc » contient 4 lignes, 4 mots et 36 caractères.

© Ahmed KHARRAT

Commandes de base (internes)

lsmod

45

R. la commande **lsmod**:

- **Syntaxe:** **lsmod**
- **Description:** **lsmod** affiche de façon agréable le contenu du fichier `/proc/modules`, montrant ainsi quels sont les modules actuellement chargés.
- Dans l'ordre :
 - 1) le nom du module,
 - 2) la quantité de mémoire qu'il utilise,
 - 3) le nombre des autres modules qui utilisent celui-ci
 - 4) et leurs noms.
- En fait, c'est une mise en forme de : `cat /proc/modules`
- La commande **lsmod** permet de lister les modules chargés par le système.

© Ahmed KHARRAT

Commandes de base (internes)

lsmod

46

R. la commande **lsmod**:

- **extrait du retour de la commande ci-dessus :**

```
ahmed@ahmed-VirtualBox:~$ lsmod
Module                  Size  Used by
nls_iso8859_1           16384  1
snd_intel8x0            49152  2
snd_ac97_codec          155648  1 snd_intel8x0
ac97_bus                16384  1 snd_ac97_codec
snd_pcm                 135168  2 snd_intel8x0,snd_ac97_codec
intel_rapl_msr          20480  0
intel_rapl_common       40960  1 intel_rapl_msr
snd_seq_midi            20480  0
snd_seq_midi_event      16384  1 snd_seq_midi
crct10dif_pclmul       16384  1
ghash_clmulni_intel    16384  0
sha256_ssse3           32768  0
sha1_ssse3              32768  0
binfmt_misc            24576  1
snd_rawmidi            49152  1 snd_seq_midi
aesni_intel            376832  0
joydev                 32768  0
crypto_simd            16384  1 aesni_intel
cryptd                 24576  2 crypto_simd,ghash_clmulni_intel
rapl                   20480  0
snd_seq                77824  2 snd_seq_midi,snd_seq_midi_event
snd_seq_device         16384  3 snd_seq,snd_seq_midi,snd_rawmidi
snd_timer              40960  2 snd_seq,snd_pcm
```

© Ahmed KHARRAT

Commandes de base (internes)

ps

47

S. la commande **ps**:

- **Syntaxe:** **ps**
- **Description:** **ps** permet d'afficher la liste des processus en cours. De nombreuses options, dépendant du Shell utilisé et de ses versions, permettent d'obtenir diverses informations : consultez le manuel.
- **Exemple:** **ps tree** permet d'afficher la liste des processus en cours d'exécution sous la forme d'un arbre.

© Ahmed KHARRAT

Commandes de base (internes)

kill

48

T. la commande **kill**:

- **Syntaxe:** **kill** nom-processus
- **Description:** Pour arrêter un processus, on utilise la commande **kill**, si on ne sait pas faire autrement.
- **Exemple:**
kill pid.
- On peut envoyer un signal plus ou moins fort. Le plus fort est:
kill pid -9.

© Ahmed KHARRAT

Commandes de base (internes)

Quelques codes de contrôle

49

- **<Control> s** : pour suspendre l'affichage d'un écran,
- **<Control> q** : pour redémarrer un affichage suspendu,
- **<Control> d** : pour indiquer la fin d'un fichier,
- **<Control> c** : pour annuler une commande,
- **<Control> z** : pour suspendre un processus (fg pour reprendre le processus; bg pour le reprendre en arrière plan).

© Ahmed KHARRAT

Commandes de base (internes)

Arrêt du système

50

- **halt** : arrêt du système.
- **reboot** : redémarrer
- **shutdown -h now** : arrêt immédiat du système.
- **shutdown -r now** : reboot immédiat du système.
- **shutdown -h 12 :40** : arrêt du système à 12h40min.
- **shutdown -r +5** : reboot dans 5min du système.
- **init 0** : arrêt immédiat du système.
- **init 6** : reboot immédiat du système.

© Ahmed KHARRAT

Application

51

1. Créer l'arborescence suivante.
2. Lister le contenu du répertoire Rep1.
3. Placer vous dans le répertoire Rep111.
4. Placer vous dans le répertoire Rep112 en utilisant un chemin relatif.
5. Copier le contenu du fichier fic1.txt dans répertoire Rep12 en gardant le même nom.
6. Afficher, avec leurs numéros, les lignes du fichier fic1.txt qui contiennent la chaîne « Shell » en ne tenant pas compte des minuscules et des majuscules.
7. Vérifiez que vous êtes dans le bon répertoire.
8. Enfin détruire le répertoire Rep12.

52

Prochain cours: **Les droits d'accès sur les répertoires et sur les fichiers**

Merci

Des questions?

© Ahmed KHARRAT